

Step by step instructions to implement resumes-ai-rag

01: Create a new folder 'resumes-ai-rag'

02: Add folder to workspace

03: Add Architecture.md file to this folder

04: Open co-pilot chat and choose 'Ask' mode and 'GPT-5-mini' model

05: Use this template to start :

'I am building Resume Search Alogrithm using AI - RAG approach and the architecture document is attached to the context. Start with creating prompt file(s), instructions that can be used by co-pilot side chat for generating / editing / debugging code for the project'

06: Drag and drop 'Architecture.md' to copilot and paste the above content and click execute.

07: Take a look at the references, debugging view to understand

08: If you are good with the prompt files, change to 'Agent' mode and ask to generate:

'I am okay with prompt files and instructions - create them now for my usage. Follow enterprise grade solutions'

09: You will notice the base prompt and templates are created

10: Click on Settings icon, you can optionally add them or else drag and drop based on need.

11. Next is to generate the code - start with agent mode {(as ask mode is done in chatgpt)}:

Generate Project scaffold + health endpoints + MongoDB connection.

12. You will notice the folders and files are generated but may show errors

13. Open the terminal and run

'npm install'

that will resolve the errors

Note: There will be errors due to non-loaded components

(package.json : mongo version : 5.7.0, if error comes like : No matching version found)

14. Click Keep button on sidebar to store them locally

15. If you still see errors — ask agent: "Fix the errors in workspace", then drag & drop those error files alone (agent will fix missing dependencies/compatibility). **Confirm errors are fixed.**

16. copy .env.example to .env file

17. Update your mongodb connection(db_url, db_name, collection_name, bm25_index, vector_index) in the .env and save it

18. Now in the terminal - do

```
'npm run dev'
```

19. Open the postman and hit healthcheck endpoint

```
http://localhost:3000/v1/health
```

20. If it is failing, ask agent to fix it by providing the message / error that you notice in postman or terminal.

21. Continue with mongodb health check

```
http://localhost:3000/v1/health/db
```

22. Change to Ask Mode. Drag and drop “Architecture.md” and “.env” file.

```
'To implement Vector Embedding + `v1/embeddings` with Architecture.md and create_endpoint.md prompt file. Confirm if you have everything to code (Do not generate code)>> (drag + drop)
```

23. Confirm references are used - note if any questions: - answer accordingly (avoid batches, avoid hardcodings)

24. Switch to Agent mode >> Create the mistral model name and api key in the '.env' file before you proceed

25. Update manually >> .env file with mistral key and dimension as 1024 and save it

```
EMBEDDING_MODEL=mistral-embed
```

```
EMBEDDING_DIM=1024
```

26. Generate code 'To implement Vector Embedding + `v1/embeddings` with Architecture.md and create_endpoint.md prompt file' with mistral-embed-small for embeddings (store in .env) as single input at time.

27. If agent updates package.json and ask you to approve 'npm install' accept those.

28. stop the running server (ctrl + c) in the terminal and run 'npm run dev' again

29. go to postman and confirm http://localhost:3000/v1/embeddings

```
{"input":"hello world"}
```

30. If all good, continue to next step - else go to debugging

31. Switch to Agent mode >> Create the groq LLM model name and api key in the '.env' file before you proceed

32. Update manually >> .env file with groq key and model name and save it

```
GROQ_API_KEY=your_groq_api_key_here
```

```
GROQ_LLM_MODEL=meta-llama/llama-4-scout-17b-16e-instruct
```

33. Click 'Keep' and Restart the server terminal

34. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'Atlas Search BM25 config + `/v1/search/bm25`.'
35. Confirm service, routes and are created and index is updated
36. Restart the server and test the following 'Debugging Notes'
37. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'Vector Search config + `/v1/search/vector`. DON'T use knn beta.
38. Confirm service, routes and are created and index is updated.
39. Restart the server and test the following 'Debugging Notes'
40. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'Hybrid Search(BM25 and vector) config + `/v1/search/hybrid`.'
41. Confirm service, routes and are created and index is updated.
42. Restart the server and test the following 'Debugging Notes'
43. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'LLM Re-Ranking config + `/v1/search/rerank`.'
44. Confirm service, routes and are created and index is updated
45. Restart the server and test the following 'Debugging Notes'
46. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'Summarization config + `/v1/search/summarize`.'
47. Confirm service, routes and are created and index is updated
48. Restart the server and test the following 'Debugging Notes'
49. Agent Mode :: Drag and drop 'Architecture.md' to co-pilot. Implement 'End-to-End Resume Search Pipeline config + `/v1/search`.'
50. Confirm service, routes and are created and index is updated
51. Restart the server and test the following 'Debugging Notes'

Debugging Notes:

- If there are issues - follow these steps:

- 1) Read the error messages (terminal, postman)
- 2) Change to Ask mode - 'Analyse the error and provide crisp updates' + error
- 3) Read the fix and if you are comfortable - fix it !